CS-GY 6033

Design and Analysis of Algorithms I

Lecture 12 | Part 1

Today's Lecture: Massive data and Complexity Theory

Massive Sets

- You've collected 1 billion tweets.[1]
- **Goal**: given the text of a new tweet, is it already in the data set?

[1] This is about two days of activity.

Membership Queries

- We want to perform a **membership query** on a collection of strings.
- Hash tables support $\Theta(1)$ membership queries.
- **Idea**: so let's use a hash table (Python: `set`).

Problem: Memory

- How much memory would a `set` of 1 billion strings require?
- Assume average string has 100 ASCII characters.

$$(8 \text{ bits per char}) \times (100 \text{ chars}) \times 1 \text{ billion} = 100 \text{ gigabytes}$$

- That's way too large to fit in memory!

Today's Lecture

- **Goal**: fast membership queries on massive data sets.
- Today's answer: **Bloom filters**.

CS-GY 6033
Design and Analysis of Algorithms I

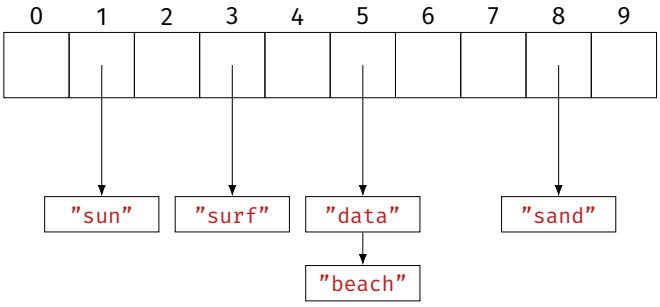
Lecture 12 | Part 2

Bit Arrays

The Challenge

- We want to perform membership queries on a massive collection (too large to fit in memory).
- We want to remember which elements are in the collection...
- ...*without* actually storing all of the elements.
- From hash tables to Bloom filters in 3 steps.

First Stop: Hash Tables

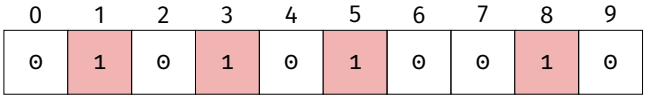


s	hash(s)
"surf"	3
"sand"	8
"data"	5
"sun"	1
"beach"	5

Memory Usage

- **Problem**: we're storing all of the elements.
- Why? To resolve collisions.
- **Fix**: ignore collisions.

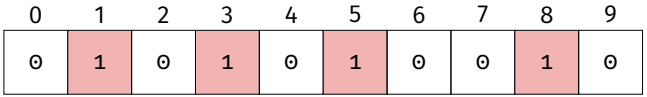
Second Stop: Hashing Into Bit Arrays



s	hash(s)
"surf"	3
"sand"	8
"data"	5
"sun"	1
"beach"	5

- Use a bit array `arr` of size c.
- **Insertion**: Set `arr[hash(x)] = 1`.
- **Query**: Check if `arr[hash(x)] = 1`.

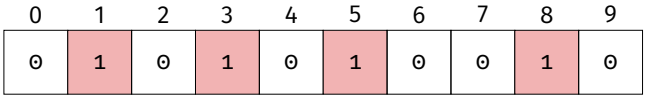
Second Stop: Hashing Into Bit Arrays



s	hash(s)
"surf"	3
"sand"	8
"data"	5
"sun"	1
"beach"	5

- Use a bit array `arr` of size c.
- **Insertion**: Set `arr[hash(x)] = 1`.
- **Query**: Check if `arr[hash(x)] = 1`.
- Can be **wrong**!

False Positives



s	hash(s)
"surf"	3
"sand"	8
"data"	5
"sun"	1
"beach"	5

- Query can return **false positives**.
- e.g., `hash("nyu") == 3`
- Cannot return false negatives.

Memory Usage

- Requires c bits, where c is size of the bit array.
- False positive rate depends on c.
  - c is small → more collisions → more errors
  - c is large → fewer collisions → fewer errors
- **Tradeoff**: get more accuracy at cost of memory.

False Positive Rate

- What is the probability of a false positive?
- Suppose there are c buckets, and we've inserted n elements so far.
- We query an object `x` that we haven't seen before.
- False positive $\Leftrightarrow$ `arr[hash(x)] == 1`.

False Positive Rate

- Assume `hash` assigns bucket uniformly at random.
  - If $x \neq y$ then, $\mathbb{P}(\texttt{hash}(\texttt{x}) = \texttt{hash}(\texttt{y})) = 1/c$
- Prob. that first element does not collide with x: $1 - 1/c$.
- Prob. that first two do not collide: $(1 - 1/c)^2$.
- Prob. that all n elements do not collide: $(1 - 1/c)^n$.

False Positive Rate

- Hint: for large z, $(1 - 1/c)^z \approx \frac{1}{e}$
- So the probability of no collision is:

$$(1 - 1/c)^n = [(1 - 1/c)^c]^{n/c} \approx e^{-n/c}$$

- This is the probability of no false positive.
- Probability of false positive upon querying x: $\approx 1 - e^{-n/c}$

False Positive Rate

- For fixed query, probability of false positive: $\approx 1 - e^{-n/c}$.
  - n: number of elements stored
  - c: size of array (number of bits)

- Randomness is over choice of hash function.
  - Once hash function is fixed, the result is always the same.

Fixing False Positive Rate

- Suppose we'll tolerate false positive rate of ε.
- Assume that we'll store around n elements.
- We can choose c:

$$1 - e^{-n/c} = \varepsilon \qquad \Longrightarrow \qquad c = -\frac{n}{\ln(1 - \varepsilon)}$$

Example

- Suppose we want ≤ 1% error.
- Previous slide says our bit array needs to be 100 times larger than number of elements stored.[2]
- Memory when $n = 10^9$: 1 billion bits ×100 = 12.5 GB.
- Can we do better?

[2] We could have guessed this, huh?

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 3

Bloom Filters

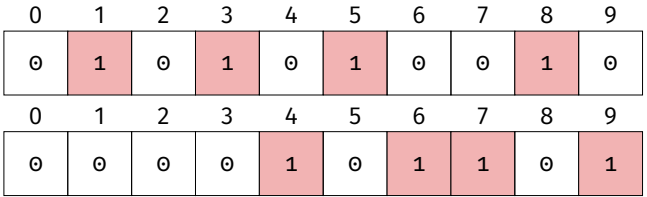
Wasted Space

- Suppose we want ≤ 1% error.
- Our bit array needs to be 100 times larger than number of elements stored.
- That's a lot of **wasted space**!

Third Stop: Multiple Hashing

- **Idea**: use several smaller bit arrays, each with own hash function.

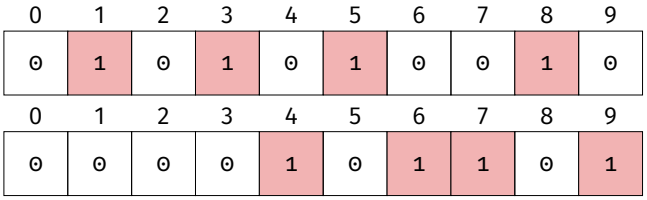
Third Stop: Multiple Hashing



s	hash_1(s)	hash_2(s)
"surf"	3	7
"sand"	8	7
"data"	5	4
"sun"	1	9
"beach"	5	6

- Use k bit arrays of size c, each with own independent hash function.
- **Insertion**: Set `arr_1[hash_1(x)] = 1`, `arr_2[hash_2(x)] = 1`, ..., `arr_k[hash_k(x)] = 1`.

Third Stop: Multiple Hashing



s	hash_1(s)	hash_2(s)
"surf"	3	7
"sand"	8	7
"data"	5	4
"sun"	1	9
"beach"	5	6

- Use *k* bit arrays of size *c*, each with own independent hash function.
- **Query**: Return `True` if **all** of `arr_1[hash_1(x)] = 1`, `arr_2[hash_2(x)] = 1`, ..., `arr_k[hash_k(x)] = 1`.
- Example:
  `hash_1("hello") == 3`,
  `hash_2("hello") == 2`

Exercise

What effect does increasing k have on false positive rate?

Intuition

- False positive occurs only if false positive in **all** tables.
- This is pretty unlikely.
- If false positive rate in one table is small (but not tiny), probability false positive in all tables is still tiny.

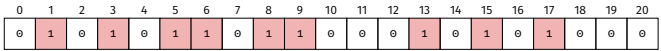
More Formally

- Probability of false positive in first table: $\approx 1 - e^{-n/c}$.
- Probability of false positive in all k tables: $\approx (1 - e^{-n/c})^k$.
- Example: if $c = 4n$ and $k = 3$, error rate is $\approx 1\%$.
- Uses only $12 \times n$ bits, as opposed to $100 \times n$ from before.

Last Stop: Bloom Filters

- How many different bit arrays do we use? (What is k?)
- How large should they be? (What is c?)
- **Bloom filters**: use k hash functions, but only one medium-sized array.

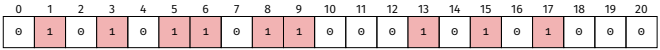
Last Stop: Bloom Filters



s	hash_1(s)	hash_2(s)
"surf"	13	17
"sand"	8	6
"data"	15	1
"sun"	1	3
"beach"	5	9

- Use one bit arrays of size *c*, but *k* hash functions.
- **Insertion**: Set `arr[hash_1(x)] = 1`, `arr[hash_2(x)] = 1`, ..., `arr[hash_k(x)] = 1`.

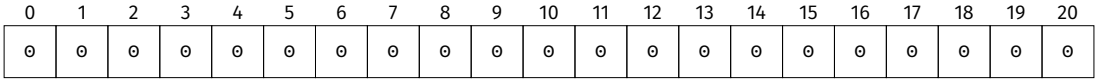
Last Stop: Bloom Filters



s	hash_1(s)	hash_2(s)
"surf"	13	17
"sand"	8	6
"data"	15	1
"sun"	1	3
"beach"	5	9

- Use one bit arrays of size *c*, but *k* hash functions.
- **Query**: Return `True` if **all** of `arr[hash_1(x)] = 1`, `arr[hash_2(x)] = 1`, ..., `arr[hash_k(x)] = 1`.
- Example:
  `hash_1("hello") == 3`,
  `hash_2("hello") == 2`

Example



s	h1(s)	h2(s)	h3(s)
"surf"	13	17	3
"sand"	8	6	19
"data"	15	3	7
"sun"	1	3	5
"beach"	13	9	11
"justin"	13	7	8

- Insert "surf".
- Insert "sand".
- Insert "data".
- Query above strings.
- Query "sun".
- Query "beach".
- Query "justin".

Intuition

- Multiple hashing allows bit arrays to be smaller.
- Even more efficient: let them share memory.
- “Overlaps” are just collisions; we can handle them.

Exercise

What effect does increasing k have on the false positive rate? Can we increase it *too high*?

Tradeoffs

- Increasing k decreases false positive rate, but only to a point.
- Eventually, k is so large that we get too many overlaps.
- At this point, false positives start to increase again.

False Positive Rate

- Consider querying new, unseen object `x`.
- We'll look at k bits.
  - `arr[hash_1(x)]`, ..., `arr[hash_k(x)]`.
- Fix one bit. What is the chance that it is already one?

False Positive Rate

- Probability of bit being zero after first element inserted: $(1 - 1/c)^k$
- After second element inserted: $(1 - 1/c)^{2k}$
- After all n elements inserted: $(1 - 1/c)^{nk}$
- And:

$$(1 - 1/c)^{nk} = [(1 - 1/c)^c]^{nk/c} \approx e^{-nk/c}$$

False Positive Rate

- Probability of bit being **one** after n elements inserted:

$$1 - e^{-nk/c}$$

- For a false positive, all k bits (for each hash function) need to be one.
- Assuming independence,[3] probability of false positive:

$$(1 - e^{-nk/c})^k$$

[3] Only true approximately. If this bit was set, some other bit was not.

Minimizing False Positives

- For a fixed n and c, the number of hash functions k which minimizes the false positive rate is

$$k = \frac{c}{n} \ln 2$$

- Plugging this into the error rate:

$$\varepsilon = (1 - e^{-nk/c})^k \qquad \Longrightarrow \qquad \ln \varepsilon = -\frac{c}{n}(\ln 2)^2$$

- If we fix ε, then $c = -n \ln \varepsilon / (\ln 2)^2$

Summary: Designing Bloom Filters

- Suppose we wish to store n elements with ε false positive rate.
- Allocate a bit array with $c = -n \ln \varepsilon / (\ln 2)^2$ bits.
- Pick $k = \frac{c}{n} \ln 2$ hash functions.

Example

- Let $n = 10^9$, $\varepsilon = 0.01$.
- We need $c \approx 9.5n \to 10n$ bits = 1.25 GB.
- We choose $k = \frac{9.5n}{n} \ln 2 \to 7$ hash functions.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 4

Bloom Filters in Practice

Applications

- A cool data structure.
- Most useful when data is huge or memory is small.

Application #1

- De-duplicate 1 billion strings, each about 100 bytes.
- Memory required for `set`: 100 gigabytes.
- Instead:
  - Loop through data, reading one string at a time.
  - If not in Bloom filter, write it to file.
- With 1% error rate, takes 1.25 GB.

Application #2

- A ***k*-mer** is a substring of length k in a DNA sequence:

  "GATTACATATAGGTGTCGA"

- Useful: does a long string have a given k-mer?

- There are a *massive* number of possible k-mers.
  - 4^k, to be precise.
  - Example: there are over 10^{18} 30-mers.

- Slide window of size k over sequence, store each substring in Bloom filter.

Application #2

- Human genome is a 725 Megabyte string, 2.9 billion characters.
- To store all k-mers, each character stored k times.
- Storing 30-mers in `set` would take 30 × 725 MB ≈ 22 GB.
- By “forgetting” the actual strings, Bloom filter (1% false positive) takes

  2.9 billion bits ≈ 360 megabytes

Application #3

- Suppose you have a massive database on disk.
- Querying the database will take a while, since it has to go to disk.
- Build a Bloom filter, keep in memory.
  - If Bloom filter says x not in database, don't perform query.
  - Otherwise, perform DB query.
- Speeds up time of "misses".

Limits

- Bloom filters are useful in certain circumstances.
- But they have disadvantages:
  - Need good idea of size, *n*, ahead of time.
  - There are false positives.
  - The elements are not stored (can't iterate over them).
- Often a `set` does just fine, with some care.

Example

- Suppose you have 1 billion tweets.
- Want to de-duplicate them by **tweet ID** (64 bit number).
- Total size: 8 gigabytes.
- I have 4 GB RAM. Should I use a Bloom filter?

De-duplication Strategy

- Design a hash function that maps each tweet ID to $\{1, \ldots, 8\}$.
- Loop through tweet IDs one-at-a-time, hash, write to file:

  `hash(x) == 3` → write to `data_3.txt`
- Read in each file, one-at-a-time, de-duplicate with `set`, write to `output.txt`

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 5

Complexity Theory

The quest for efficient algorithms is about finding clever ways to avoid taking exponential time. So far we have seen the most brilliant successes of this quest; now we meet the quest's most embarrassing and persistent failures.

- paraphrased from *Algorithms* by Dasgupta, Papadimitriou, Vazirani

Exponential to Polynomial

- Many problems have brute force solutions which take exponential time.
- Example: clustering to maximize separation
- The challenge of algorithm design: find a more “efficient” solution.

Polynomial Time

- If an algorithm's worst case time complexity is $O(n^k)$ for some k, we say that it runs in **polynomial time**.
  - Example: $\Theta(n \log n)$, since $n \log n = O(n^2)$.
- Any polynomial is much faster than exponential for big n.
  - But not necessarily for small n.
  - Example: n^{100} vs 1.0001^n.
- We therefore think of polynomial as "efficient".

Question

- Is every problem solvable in polynomial time?

Question

- Is every problem solvable in polynomial time?
- **No!** Problem: print all permutations of n numbers.

Ok, then...

- What problems can be solved in polynomial time?
- What problems can’t?
- How can I tell if I have a hard problem?

Ok, then...

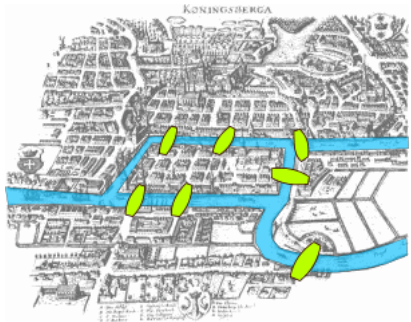
- What problems can be solved in polynomial time?
- What problems can't?
- How can I tell if I have a hard problem?
- Core questions in **computational complexity theory**.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 6

Eulerian and Hamiltonian Cycles

Example: Bridges of Königsberg

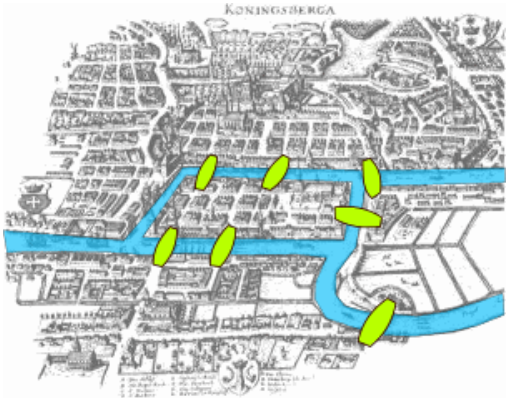


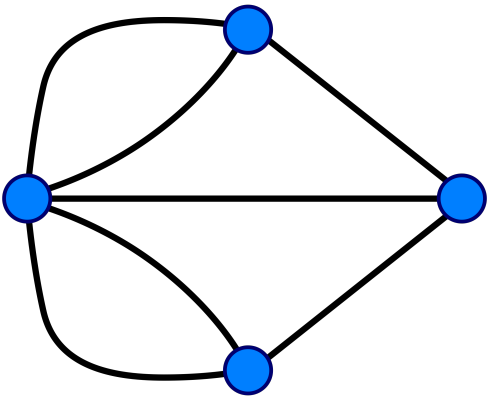
- **Problem**: Is it possible to start and end at same point while crossing each bridge exactly once?

Leonhard Euler

1707 - 1783

Eulerian Cycle

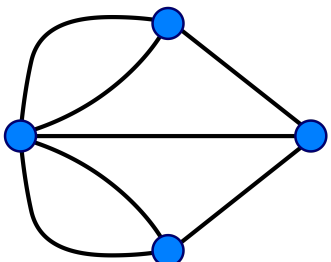




Is there a cycle which uses each edge exactly once?

Necessary conditions

- Graph must be connected.
- Each node must have even degree.
- Answer for Königsberg answer: it is **impossible**.



In General...

- These conditions are **necessary** and **sufficient**.

- A graph has a Eulerian cycle **if and only if**:
  - it is connected;
  - each node has even degree.

Exercise

Can we determine if a graph has an Eulerian cycle in time that is polynomial in the number of nodes?

Remember, an Eulerian cycle exists iff the graph is connected and each node has even degree.

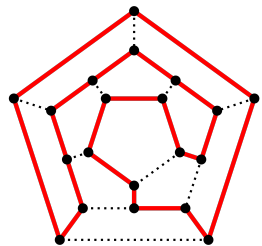
Answer

- We can check if it is connected in $\Theta(V + E)$ time.
- Compute every node's degree in $\Theta(V)$ time with adjacency list.
- Total: $\Theta(V + E) = O(V^2)$. **Yes!**

Hamiltonian Cycles

- A **Hamiltonian cycle** is a cycle which visits each *node* exactly once (except the starting node).

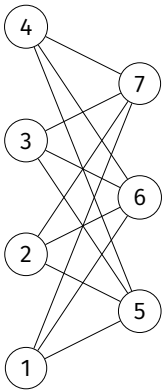
- Example: find a Hamiltonian cycle on the graph below:



Exercise

Can we determine whether a general graph has a Hamiltonian cycle in polynomial time?

Some cases are easy



In General

- Could brute-force.
- How many possible cycles are there?

Hamiltonian Cycles are Difficult

- This is a **very difficult** problem.
- No polynomial algorithm is known for general graphs.
- In special cases, there may be a fast solution. But in general, worst case is hard.

Note

- Determining if a graph has a Hamiltonian cycle is **hard**.
- But if we're given a "hint" (i.e., $(v_1, v_2, \ldots, v_n)$ is possibly a Hamiltonian cycle), we can check it very quickly!
- Hard to solve; but easy to verify "hints".

Similar Problems

- Eulerian: polynomial algorithm, **“easy”**.
- Hamiltonian: no polynomial algorithm known, **“hard”**.

Main Idea

Computer science is littered with pairs of similar problems where one easy and the other very hard.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 7

Shortest and Longest Paths

Problem: SHORTPATH

- **Input:** Graph[4] G, source u, dest. v, number k.
- **Problem:** is there a path from u to v of length $\leq k$?
- **Solution:** BFS or Dijkstra/Bellman-Ford in polynomial time.
- **Easy!**

[4] Weighted with no negative cycles, or unweighted.

Problem: LongPath

- **Input:** Graph[5] G, source u, dest. v, number k.
- **Problem:** is there a **simple** path from u to v of length $\geq k$?
- Naïve solution: try all $V!$ path candidates.

[5] Weighted or unweighted.

Long Paths

- There is no known polynomial algorithm for this problem.
- It is a **hard problem**.
- But given a “hint” (a possible long path), we can verify it very quickly!

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 8

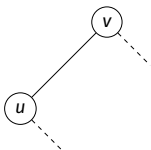
Reductions

Reductions

- HAMILTONIAN and LONGPATH are related.
- We can “convert” HAMILTONIAN into LONGPATH in polynomial time.
- We say that HAMILTONIAN **reduces** to LONGPATH.

Reduction

- Suppose we have an algorithm for LongPath.
- We can use it to solve Hamiltonian as follows:



  - Pick arbitrary node u.
  - For each neighbor v of u:
    - Create graph G' by copying G, deleting (u, v)
    - Use algorithm to check if a simple path of length $\geq |V| - 1$ from u to v exists in G'.
    - If yes, then there is a Hamiltonian cycle.

Reductions

- If Problem A reduces[6] to Problem B, it means “we can solve A by solving B”.
- Best possible time for A ≤ best possible time for B + polynomial
- “A is no harder than B”
- “B is at least as hard as A”

[6] We’ll assume reduction takes polynomial time.

Relative Difficulty

- If Problem *A* reduces to Problem *B*, we say *B* is **at least as hard** as *A*.
- Example: HAMILTONIAN reduces to LONGPATH. LONGPATH is at least as hard as HAMILTONIAN.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 9

$P \stackrel{?}{=} NP$

Decision Problems

- All of these problems are **decision problems**.
  - Output: yes or no.
  - Example: Does the graph have an Euler cycle?

P

- Some problems have polynomial time algorithms.
  - ShortPath, Euler
- The set of decision problems that can be solved in polynomial time is called **P**.
- Example: ShortPath and Euler are in P.

NP

- The set of decision problems with “hints” that can be verified in polynomial time is called **NP**.
- All of today’s problems are in NP.
  - All problems in P are also in NP.
- Example: ShortPath, Euler, Hamiltonian, LongPath are all in NP.

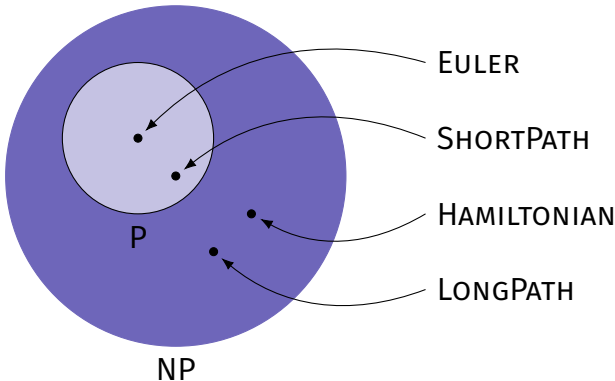
$\mathbf{P} \subset \mathbf{NP}$

- P is a subset of NP.
- It *seems* like some problems in NP aren't in P.
  - Example: HAMILTONIAN, LONGPATH.
  - We don't know polynomial time algorithms for these problems.
- But that doesn't mean such an algorithm is impossible!

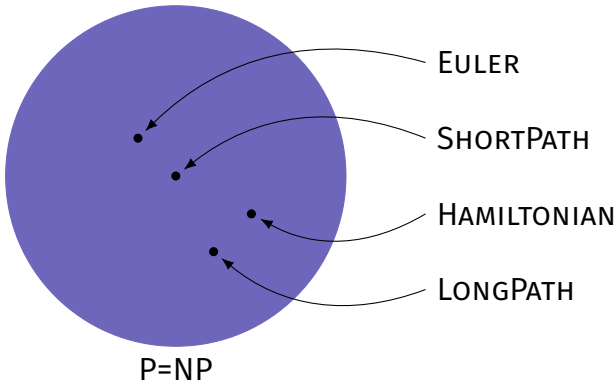
P = NP?

- Are there problems in NP that aren't in P?
  - That is, is P ≠ NP?
- Or is any problem in NP also in P?
  - That is, is P = NP?

P ≠ NP



P = NP



P = NP?

- Is P = NP?

[7] If you solve it, you'll be rich and famous.

P = NP?

- Is P = NP?
- **No one knows!**
- Biggest open problem in Math/CS.[7]
- Most think P ≠NP.

[7] If you solve it, you'll be rich and famous.

What if P = NP?

- Possibly Earth-shattering.
  - Almost all cryptography instantly becomes obsolete;
  - Logistical problems solved exactly, quickly;
  - *Mathematicians* become obsolete.

- But maybe not...
  - Proof could be non-constructive.
  - Or, constructive but really inefficient. E.g., $\Theta(n^{10000})$

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 10

NP-Completeness

Problem: 3-SAT

- Suppose `x_1, ..., x_n` are boolean variables (`True`,`False`)
- A **3-clause** is a combination made by `or`-ing and possibly negating three variables:
  - `x_1 or x_5 or (not x_7)`
  - `(not x_1) or (not x_2) or (not x_4)`

Problem: 3-SAT

- **Given:** *m* clauses over *n* boolean variables.
- **Problem:** Is there an assignment of $x_1, \ldots, x_n$ which makes all clauses true simultaneously?
- No polynomial time algorithm is known.
- But it is easy to verify a solution, given a hint.
  - 3-SAT is in NP.

Cook's Theorem

Every problem in NP is polynomial-time reducible to 3-SAT.

- ...including Hamiltonian, long path, etc.
- 3-SAT is at least as hard as every problem in NP.
- "hardest problem in NP"

Cook's Theorem (Corollary)

- If 3-SAT is solvable in polynomial time, then all problems in NP are solvable in polynomial time.
  - ...including Hamiltonian, long path, etc.

NP-Completeness

- We say that a problem is **NP-complete** if:
  - it is in NP;
  - every problem in NP is reducible to it.
- HAMILTONIAN, LONGPATH, 3-SAT are all NP-complete.
- NP-complete problems are the “hardest” in NP.

Equivalence

- In some sense, NP-complete problems are equivalent to one another.
- E.g., a fast algorithm for HAMILTONIAN gives a fast algorithm for 3-SAT, LONGPATH, and all problems in NP.

Who cares?

- Complexity theory is a fascinating piece of science.
- But it's practically useful, too, for recognizing hard problems when you stumble upon them.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 12 | Part 11

Hard Optimization Problems

Hard Optimization problems

- NP-completeness refers to **decision problems**.
- What about optimization problems?
- We can typically state a similar decision problem.
- If that decision problem is hard, then optimization is at least as hard.

Problem: bin packing

- Optimization problem:
  - **Given:** bin size B, n objects of size $\alpha_1, \ldots, \alpha_n$..
  - **Problem:** find minimum number of bins k that can contain all n objects.

- Decision problem version:
  - **Given:** bin size B, n objects of size $\alpha_1, \ldots, \alpha_n$, integer k.
  - **Problem:** is it possible to pack all n objects into k bins?

- Decision problem is NP-complete, reduces to optimization problem.

Example: traveling salesperson

- Optimization problem:
  - **Given:** set of n cities, distances between each.
  - **Problem:** find shortest Hamiltonian cycle.

- Decision problem:
  - **Given:** set of n cities, distance between each, length ℓ.
  - **Problem:** is there a Hamiltonian cycle of length $\leq \ell$?

- Decision problem is NP-complete, reduces to optimization problem.

NP-complete problems in machine learning

- Many machine learning problems are NP-complete.
- Examples:
  - Minimizing k-means objective.

So now what?

- Just because a problem is NP-Hard, doesn't mean you should give up.
- Usually, an approximation algorithm is fast, "good enough".
- Some problems are even hard to *approximate*.

Summary

- Not every problem can be solved efficiently.
- Computer scientists are able to categorize these problems.

CS-GY 6033
Design and Analysis of Algorithms I

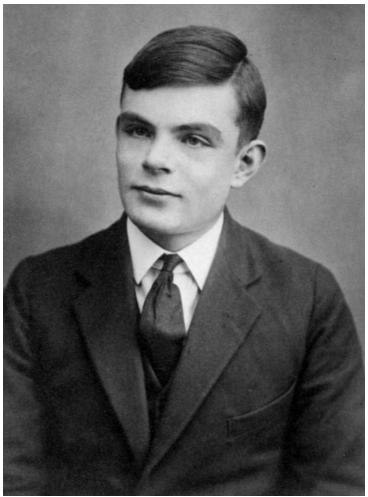
Lecture 12 | Part 12

The Halting Problem

Really hard problems

- Some decision problems are harder than others.
- That is, it takes more time to solve them.
- Given enough time, all decision problems can be solved, right?

Alan Turing



1912-1954

Turing's Halting Problem

- **Given:** a function `f` and an input `x`.
- **Problem:** does `f(x)` halt, or run forever?
- Algorithm must work for all functions/inputs!

Turing's Argument

- Turing says: no such algorithm can exist.

- Suppose there is a function `halts(f, x)`:
  - Returns `True` if `f(x)` halts.
  - Returns `False` if `f(x)` loops forever.

Turing's Argument

```
def evil_function(f):
    if halts(f, f):
        # loop forever
    else: # it runs forever
        return
```

- Consider `evil_function(evil_function)`.
  - Does it halt or not?

Turing's Argument

```
def evil_function(f):
    if halts(f, f):
        # loop forever
    else: # it runs forever
        return
```

- Consider `evil_function(evil_function)`.
  - Does it halt or not?
- Assuming that `halt` works leads to logical impossibility!
  - So a working `halt` cannot exist.

Undecidability

- The halting problem is **undecidable**.
- Fact of the universe: there can be no algorithm for solving it which works on all functions/inputs.
- All of these problems are undecidable:
  - Does the program terminate?
  - Does this line of code ever run?
  - Does this function compute what its specification says?
  - Many others...

Reality

- **Computer science:**
  - There's a speed limit for certain problems, too.
  - And some problems can't even be solved!

The End